**T.C**

**KASTAMONU ÜNİVERSİTESİ**

**MÜHENDİSLİK VE MİMARLIK FAKÜLTESİ**

**BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**DERS KODU VE DERS ADI**
**BSM204 - NESNEYE YÖNELİK PROGRAMLAMA**

**PROJE ADI**
# LotusOS: 32-Bit Grafik Arayüzlü Bir İşletim Sistemi Çekirdeği ve Masaüstü Ortamı Tasarımı

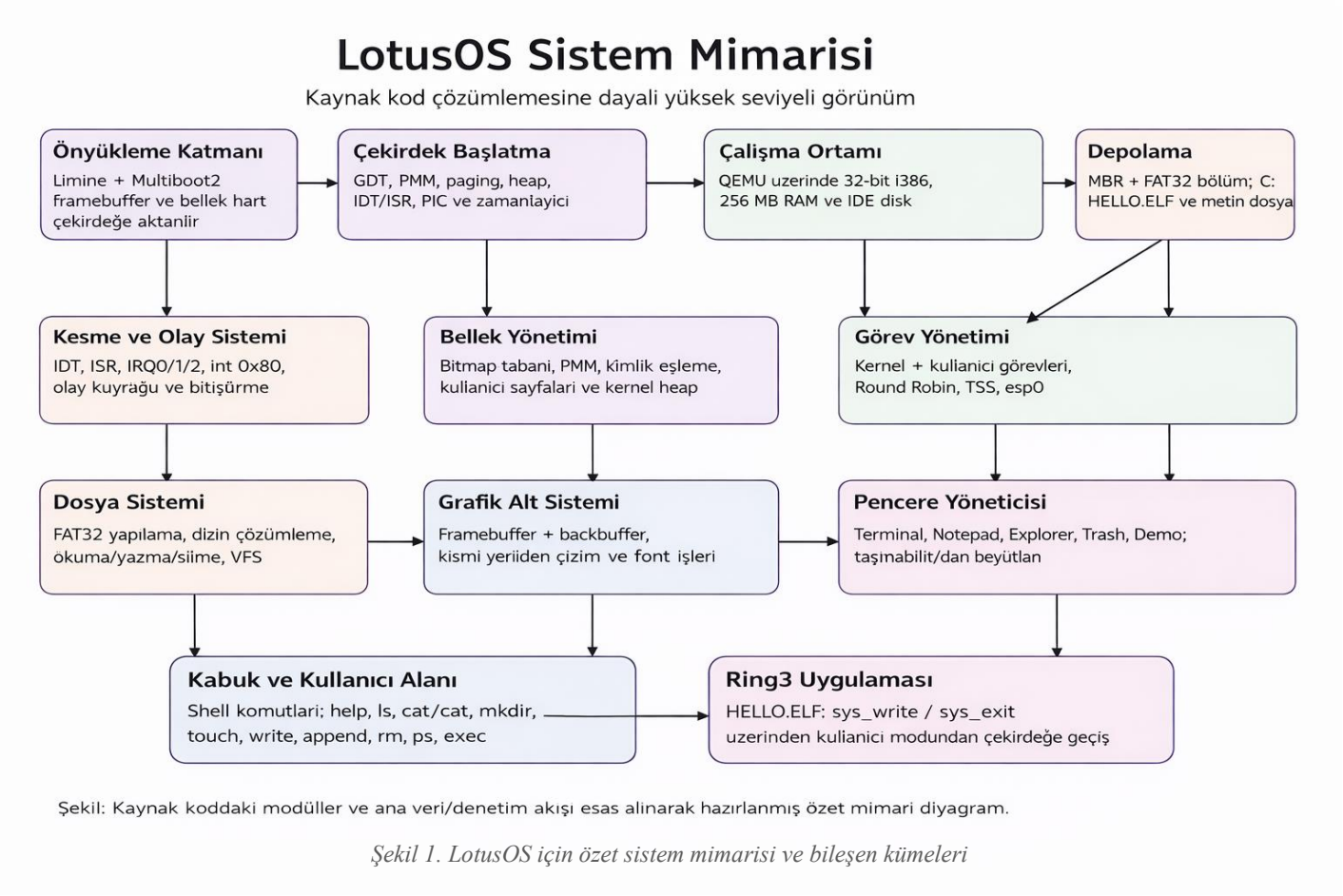
**PROJE SAHİBİ**
**244410114 RODİNA AMR MUKHTAR**

**KASTAMONU**
**29 Mart - 2026**

# LOTUSOS

*32-Bit Grafik Arayüzlü, FAT32 Destekli ve Çok Görevli Bir İşletim Sistemi Tasarımı ve Uygulama Analizi*

---




*Şekil 1. LotusOS için özet sistem mimarisi ve bileşen kümeleri*

*Not: Arşiv kök dizini LotusOS adını taşımakta; bazı eski dosya veya açıklamalarda LeiurusOS/MyOS izleri korunmaktadır. Bu raporda bütünlük için LotusOS adı esas alınmıştır [8].*

# 2. Özet

Bu rapor, LotusOS adlı deneysel işletim sistemi projesini kaynak kod, derleme hattı ve sağlanan QEMU ekran görüntüleri üzerinden akademik olarak incelemek amacıyla hazırlanmıştır. LotusOS; Limine ve Multiboot2 tabanlı önyükleme, bitmap temelli fiziksel bellek yönetimi, kimlik eşlemeli sayfalama, TSS destekli kullanıcı modu geçişi, 600 Hz zamanlayıcı ile çalışan çok görevli yapı, FAT32 tabanlı dosya sistemi ve grafik masaüstü bileşenlerini tek bir monolitik çekirdek içinde bir araya getirmektedir [3]-[8].

İnceleme sonucunda projenin yalnızca önyüklenebilir bir çekirdek üretmekle kalmadığı; aynı zamanda kabuk komutları, dosya oluşturma/düzenleme, pencere yönetimi, grafik çizim, olay kuyruğu, kullanıcı modu uygulaması ve dosya gezgini gibi gerçek işletim sistemi alt sistemlerini bir araya getirdiği görülmektedir [8]. Özellikle kullanıcı sayfaları için PDE.USER bitinin doğru biçimde yayılması, framebuffer bölgesinin fiziksel bellek yöneticisinden korunması ve olay kuyruğunda mouse/tick birleştirme mantığı gibi ayrıntılar, projenin ders düzeyini aşan mühendislik hassasiyetine sahip olduğunu göstermektedir [3], [7], [8].

## Abstract

This report presents an academic analysis of LotusOS, an experimental 32-bit operating system studied through its source archive, build pipeline, and supplied QEMU screenshots. The system combines Limine/Multiboot2 bootloading, bitmap-based physical memory management, identity-mapped paging, TSS-backed Ring3 transitions, a multitasking layer driven by a 600 Hz timer, a FAT32-based storage stack, and a graphical desktop environment inside a monolithic kernel design [3]-[8].

The evaluation shows that LotusOS goes beyond a minimal bootable kernel. It integrates a command shell, file operations, a window manager, framebuffer graphics, event queue coalescing, a user-space ELF program, and a simple desktop workflow [8]. From an educational and engineering perspective, the project is notable for exposing low-level subsystems in a coherent manner while still providing a visible end-user interface.

# 3. Giriş

## 3.1. Projenin Amacı

LotusOS projesinin temel amacı, işletim sistemi derslerinde teorik olarak ele alınan önyükleme, kesme yönetimi, bellek yönetimi, görev zamanlama ve dosya sistemi gibi çekirdek konularını doğrudan çalışan bir kod tabanı içinde görünür kılmaktır [1], [2], [8]. Projenin ayırt edici yanı, bu düşük seviyeli mekanizmaları yalnızca metin tabanlı bir konsolda bırakmayıp grafik masaüstü, pencere yönetimi ve küçük kullanıcı uygulamalarıyla somut bir etkileşim katmanına dönüştürmesidir [8].

## 3.2. Kapsam

Sistem 32-bit i386 hedefi için tasarlanmıştır. Kapsam dâhilindeki başlıca özellikler; Limine tabanlı önyükleme, Multiboot2 bilgi aktarımı, PMM + paging altyapısı, IDT/ISR/PIC yönetimi, 600 Hz zamanlayıcı, Round Robin görev sistemi, FAT32 + VFS depolama katmanı, kabuk komutları, ELF yükleyici ve grafik tabanlı masaüstü bileşenleridir [3]-[8]. Ağ yığını, SMP, demand paging, gelişmiş güvenlik modeli ve tam süreç yalıtımı gibi ileri özellikler ise mevcut sürümün kapsamı dışındadır. Bu sınırlandırma, projeyi ders ölçeğinde uygulanabilir tutarken mimari bütünlüğü korumaktadır [1], [2], [8].

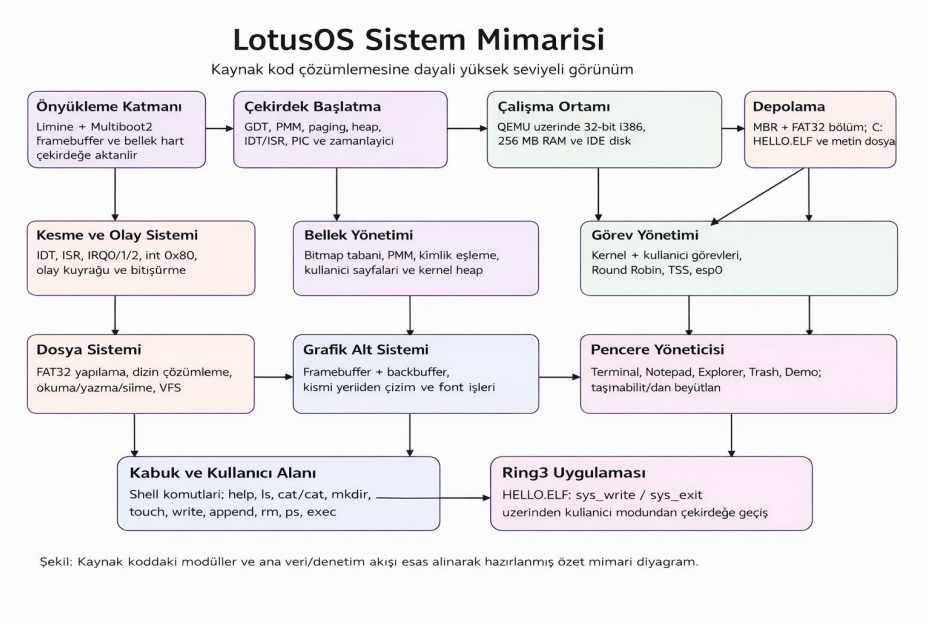
## 3.3. Kullanılan Teknolojiler

| Katman | Kullanılan araç / teknoloji | Rapor içindeki işlevi |
|---|---|---|
| Programlama dilleri | C, C++, Assembly | Çekirdek, sürücüler, pencere yöneticisi ve önyükleme katmanı |
| Derleme zinciri | GCC, LD, NASM, Make | Kernel ve kullanıcı uygulamasının derlenmesi ve bağlanması |
| Önyükleme | Limine + Multiboot2 | Kernel.elf dosyasının boot bilgileriyle yüklenmesi |
| Sanallaştırma | QEMU (32-bit i386, 256 MB RAM) | Çalıştırma ve gözlemsel test ortamı |
| Depolama | MBR + FAT32 + VFS | C: sürücüsünün bağlanması, ELF ve metin dosyalarının erişimi |
| Grafik | Framebuffer + backbuffer | Masaüstü, pencere ve kısmi yeniden çizim altyapısı |

Bu teknoloji seti, kuramsal işletim sistemi kavramları ile gözle görülebilir kullanıcı çıktısı arasında doğrudan bir bağ kurmaktadır. Özellikle Limine/Multiboot2 ile çekirdeğe zengin boot bilgisi aktarılması ve QEMU ile tekrarlanabilir test yapılması, ders projeleri için güçlü bir geliştirme zemini oluşturmaktadır [4], [5], [8].

# 4. Sistem Mimarisi (System Architecture)

Rapor formatında belirtildiği üzere bu bölüm çalışmanın merkezini oluşturmaktadır. LotusOS kaynak kodu incelendiğinde, bütün ana alt sistemlerin tek çekirdek imgesinde birleştirildiği, kullanıcı alanı ile çekirdek arasındaki geçişlerin ise TSS ve sistem çağrıları üzerinden kurulduğu monolitik bir tasarım görülmektedir [3], [8]. Bu tercih; veri yapılarının ortak paylaşımı, hata ayıklamanın sadeleşmesi ve ders ortamında alt sistem etkileşiminin görünür tutulması açısından rasyoneldir [1], [2].



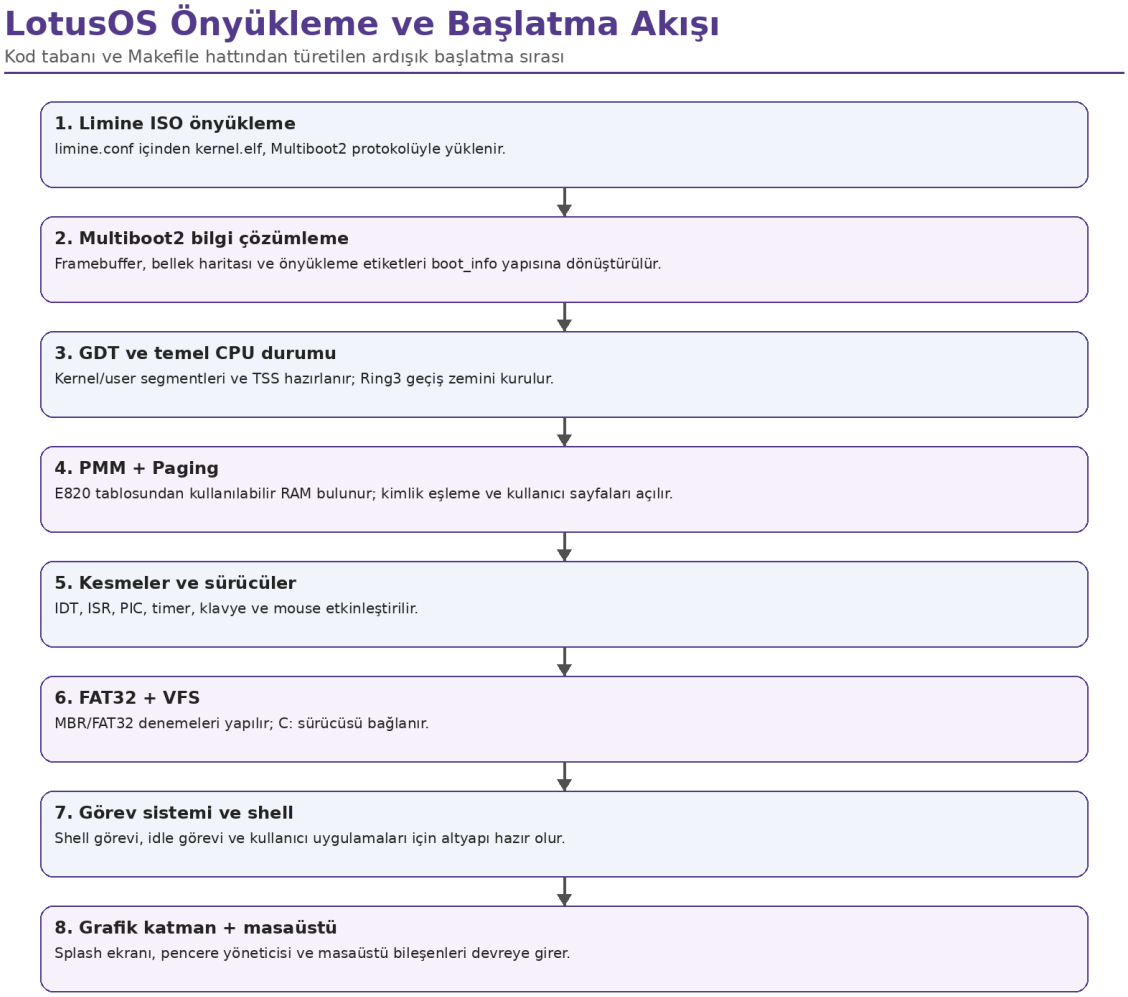

*Şekil 2. LotusOS kaynak kodundan türetilen yüksek seviyeli sistem mimarisi*

## 4.1. Çekirdek Yapısı

LotusOS çekirdeği monolitik yapıdadır. PMM, paging, kheap, FAT32/VFS, görev zamanlayıcı, kabuk ve GUI modülleri aynı çekirdek bağlamı içinde çalışmakta; sistem çağrıları ise ayrı bir kullanıcı modu yürütümünü desteklemekle birlikte çekirdek servislerine merkezi bir geçiş noktası sağlamaktadır [3], [8]. Monolitik çekirdek yaklaşımı, mikro-çekirdeğe göre daha düşük geçiş maliyeti ve daha basit veri paylaşımı sunduğu için eğitim amaçlı sistemlerde sık tercih edilen bir yaklaşımdır [1], [2].

## 4.2. Önyükleme Süreci

Dağıtımın etkin önyükleme yolu Limine tabanlıdır. Makefile, kernel.elf dosyasını üretmekte; ardından limine.conf ve gerekli BIOS önyükleme bileşenleriyle ISO imajı oluşturmaktadır. Sistem açıldığında Limine, Multiboot2 etiketleri aracılığıyla framebuffer ve bellek haritası bilgilerini çekirdeğe iletmektedir [4], [5], [8]. Bu tasarım, ham önyükleyici koduna kıyasla daha güvenilir bir yükleme zemini sunarken çekirdeğin erken aşamada ihtiyaç duyduğu donanım bilgisini standart biçimde sağlamaktadır [4], [5].

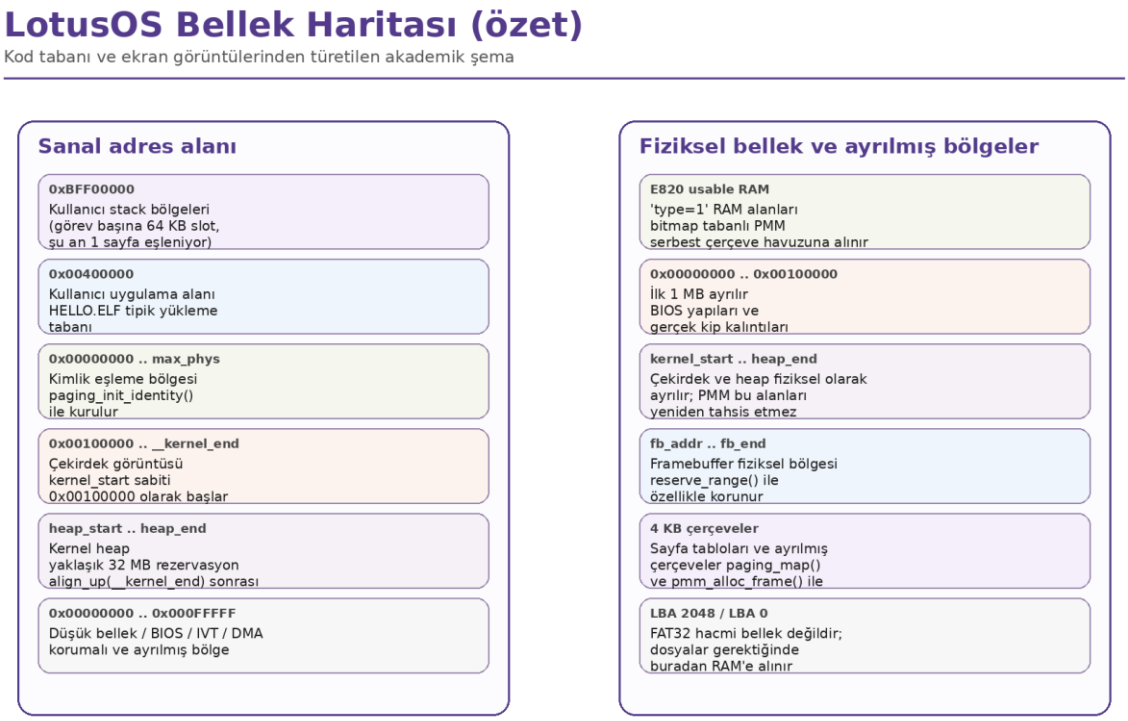



*Şekil 3. LotusOS önyükleme ve başlatma akışının adım adım özeti*

## 4.3. Bellek Yönetimi

Bellek yönetimi iki katmandan oluşmaktadır. Birinci katman, E820 bellekh haritasından gelen kullanılabilir RAM bölgelerini bitmap üzerinde izleyen fiziksel bellek yöneticisidir. İkinci katman, paging_init_identity() ile çekirdeğin kimlik eşlemeli adres alanını açan ve gerektiğinde kullanıcı sayfalarını ekleyen sayfalama altyapısıdır [3], [7], [8]. Kodda MAX_PHYS_MEM üst sınırı 512 MB olarak tanımlanmış; heap bölgesi ise çekirdek sonundan sonra yaklaşık 32 MB rezervasyon mantığıyla kurulmuştur [8].

Bu mimaride özellikle iki nokta akademik olarak önemlidir: (i) framebuffer fiziksel bölgesinin PMM tarafından tekrar tahsis edilmemesi için açıkça reserve_range() ile ayrılması ve (ii) kullanıcı sayfaları mevcut bir PDE içine yerleştirildiğinde PAGE_USER bitinin PDE düzeyinde de işaretlenmesidir. Bu ikinci ayrıntı, Ring3 tarafındaki koruma ihlallerini engelleyen kritik bir düzeltmedir [3], [7], [8].




*Şekil 4. LotusOS için özet sanal/fiziksel bellek haritası*

## 4.4. Ana Başlatma Sırası

Kod akışı incelendiğinde çekirdeğin ana başlatma sırası genel olarak gdt_init() -> pmm_init() -> paging_init_identity() -> term_init() -> kheap_init() -> idt_init() -> isr_init() -> pic_remap() -> timer_init(600 Hz) -> events_init() -> keyboard_init() -> FAT32/VFS bağlama -> task_init() -> shell_init() -> mouse_init() -> gfx_init() -> splash_show() -> wm_init() biçimindedir [8]. Bu sıranın mantığı, önce korumalı ve kararlı yürütüm zemininin kurulması, sonra donanım etkileşiminin açılması ve en sonda kullanıcıya görünür GUI katmanının devreye alınmasıdır. Böylece olası erken başlatma hataları grafik katmandan önce izole edilebilmektedir [1], [2], [8].

# 5. Temel Bileşenler ve Tasarım Kararları

## 5.1. Süreç / Görev Yönetimi

Ne yapıldı? LotusOS, görev sayısını TASK_MAX = 16 ile sınırlayan, görev başına 16 KB kernel stack ayıran ve kullanıcı görevleri için 0xBFF00000 tabanlı ayrı stack bölgeleri tanımlayan bir çok görevli yapı kurmaktadır [8]. Shell görevi sistem açılışında çalışan ilk bağlam olarak tutulmakta, ayrıca idle görevi ve kullanıcı modu görevleri aynı zamanlayıcı içinde işletilmektedir [8].

Neden bu yaklaşım seçildi? Round Robin, ders ölçeğindeki sistemlerde uygulanması kolay, adil ve öngörülebilir bir zamanlama yaklaşımıdır. Öncelik tabanlı veya çok seviyeli geri beslemeli kuyruklar daha güçlü olmakla birlikte açlık, karmaşık politika ayarı ve daha zor hata ayıklama maliyeti doğurur [1], [2]. LotusOS kodunda quantum değeri 5 timer tick olarak tanımlanmış, timer frekansı ise 600 Hz seçilmiştir; bu da yaklaşık 8.33 ms'lik bir zaman dilimine karşılık gelmektedir [8]. Bu değer, hem GUI tepkiselliği hem de bağlam değiştirme maliyeti arasında dengeli bir tercih oluşturmaktadır.

Nasıl gerçekleştirildi? task_maybe_switch() fonksiyonu önce görev ölümü ve gönüllü yield isteklerini denetlemekte, ardından yalnızca timer IRQ'si sırasında quantum sayaçlarını ilerletmektedir. Yeni göreve geçiş anında TSS.esp0 güncellenmekte, böylece Ring3 -> Ring0 geçişlerinde doğru kernel stack kullanılmaktadır [3], [8].

## 5.2. Kesme Yönetimi (Interrupt Handling)

Ne yapıldı? Sistem, IDT tablosu, assembly ISR sarmalayıcıları ve C tarafındaki isr.c mantığı üzerinden işlemci istisnalarını, donanım IRQ'larını ve int 0x80 sistem çağrısı kapısını karşılamaktadır [3], [7], [8]. PIC, 0x20 ve 0x28 vektör tabanlarına yeniden eşlenmiş; böylece klasik IRQ hatları işlemci istisnalarıyla çakışmadan kullanılabilir hâle getirilmiştir [8].

Neden bu yaklaşım seçildi? Ayrık IDT + ISR + PIC modeli, x86 mimarisinin klasik ve öğretici düzenidir. Bu yapı sayesinde istisna vektörleri, zamanlayıcı, klavye, mouse ve kullanıcı çağrıları aynı kesme çatısı altında incelenebilir [3], [7]. Ayrıca olay kuyruğunda mouse hareketleri ve timer tick'leri için birleştirme yapılması, gereksiz yeniden çizimi ve kuyruk taşmasını azaltan önemli bir tasarım kararıdır [8].

Nasıl gerçekleştirildi? isr.c tarafı, int 0x80 geldiğinde syscall_dispatch() fonksiyonunu çağırmakta; klavye ve mouse olayları ise events.c içindeki kuyruk yapısına aktarılmaktadır. GUI tarafında bu kuyruk batch mantığıyla tüketildiği için hem shell hem de masaüstü katmanı aynı olay akışını paylaşabilmektedir [8].

## 5.3. Dosya Sistemi (File System)

Ne yapıldı? LotusOS depolama katmanı, FAT32 sürücüsü ile VFS soyutlamasını birlikte kullanmaktadır. Kod, önce MBR'den FAT32 bölüm aramakta; başarısız olursa bu projede yaygın kullanılan LBA 2048 ofsetini ve ardından LBA 0 süperfloppy durumunu denemektedir. Başarılı bağlama sonrası sürücü C: harfiyle VFS'ye eklenmektedir [6]-[8].

Neden FAT32 tercih edildi? FAT32, ders projeleri için son derece uygun bir seçimdir; çünkü disk üzerindeki temel veri yapıları iyi belgelenmiştir, basit araçlarla üretilebilir ve metin/ELF dosyalarıyla hızlı test yapılmasına izin verir [6], [7]. ext2 veya NTFS gibi alternatifler daha güçlü olmakla birlikte daha karmaşık meta veri yönetimi ve daha ağır yazma tutarlılığı yükü getirir. LotusOS'un amacı taşınabilir, öğretici ve doğrudan doğrulanabilir bir dosya sistemi zinciri kurmak olduğundan FAT32 burada rasyonel bir tercihtir [6], [8].

Nasıl gerçekleştirildi? FAT32 sürücüsü kök dizin ve alt dizin çözümlemesi, dosya okuma/yazma/silme ve temel dizin işlemlerini yürütmektedir. Kabuk tarafındaki ls, cat, mkdir, touch, write, append ve rm komutları bu depolama katmanını görünür şekilde test etmektedir [8].

## 5.4. Sistem Çağrıları (System Calls)

Ne yapıldı? Kullanıcı modu örnek uygulaması HELLO.ELF, int 0x80 yoluyla sys_write ve sys_exit çağrılarını yapmaktadır. Çekirdek tarafında ise syscall_dispatch() fonksiyonu eax içindeki numaraya göre hizmet seçerek yazma, çıkış ve yield işlemlerini yürütmektedir [3], [8].

Neden bu yaklaşım seçildi? Ayrı kullanıcı uygulamasıyla gerçek bir çekirdek servisi zinciri kurmak, işletim sisteminin yalnızca tek adres uzayında çalışan fonksiyonlardan ibaret olmadığını göstermesi açısından çok değerlidir. int 0x80 mekanizması, x86 sistem çağrısı öğretiminde klasik bir geçiş biçimi olduğundan ders bağlamına uygundur [3], [7].

Nasıl gerçekleştirildi? HELLO.ELF diskten okunup ELF yükleyici ile belleğe taşınmakta; ardından task_create_user() ile yeni Ring3 görevi başlatılmaktadır. Böylece dosya sistemi, ELF yükleme, görev sistemi ve kesme tabanlı sistem çağrısı aynı senaryoda birleşmektedir [8].

## 5.5. Grafik ve Masaüstü Katmanı

LotusOS'un en ayırt edici yönü grafik masaüstü katmanıdır. Sistem, framebuffer üzerine doğrudan çizim yapan grafik modülü, arka tampon kullanan sunum mantığı, ikon/dock/tabanlı masaüstü ve pencere yöneticisi ile etkileşimli bir kullanıcı arayüzü kurmaktadır [8]. Bu yapı, ders projelerinde sık görülmeyen bir bütünleşme düzeyi sunar; çünkü yalnızca kernel servisleri değil, bunların kullanıcıya nasıl sunulduğu da görünür hâle gelmektedir.

# 6. Uygulama ve Kod Yapısı

## 6.1. Klasör Yapısı

| Dizin / dosya grubu | İşlevi |
|---|---|
| boot/ | Eski stage1/stage2 önyükleme denemeleri ve limine.conf |
| kernel/ | Çekirdek, sürücüler, bellek yönetimi, GUI ve pencere yöneticisi modülleri |
| apps/ | HELLO.ELF kullanıcı uygulamasının kaynakları ve bağlayıcı betiği |
| files/ | FAT32 hacmine kopyalanan örnek dosyalar (HELLO.ELF, HELLO.TXT, README.TXT) |
| build/ | Kernel, ISO, disk imajı ve derlenmiş nesneler |
| Makefile | Bütün derleme, disk hazırlama ve ISO paketleme hattı |

## 6.2. Modül Dağılımı ve Yaklaşık Yoğunluk

| Modül | Yaklaşık satır sayısı | Yorum |
|---|---|---|
| wm.cpp | 2240 | Pencere yöneticisi, masaüstü, dock ve uygulama mantığı |
| fat32.c | 1370 | Dosya sistemi okuma/yazma ve dizin işlemleri |
| gfx.cpp | 805 | Framebuffer çizimi ve sunum katmanı |
| shell.c | 599 | Kabuk komutları ve kullanıcı etkileşimi |
| task.c | 354 | Görev yönetimi ve bağlam değiştirme |
| kernel.c | 302 | Ana başlatma akışı ve olay döngüsü |
| paging.c + pmm.c | 265 | Sanal/fiziksel bellek yönetiminin çekirdeği |

Bu tablo, projenin ağırlık merkezinin yalnızca önyükleme ve bellek tarafında değil; özellikle GUI, dosya sistemi ve kullanıcı etkileşimi katmanlarında yoğunlaştığını göstermektedir. Dolayısıyla LotusOS'un pedagojik değeri, "kernel ayağa kalkıyor" düzeyinin ötesine geçmekte ve işletim sistemi hizmetlerinin kullanıcıya yansıyan yüzünü de kapsamaktadır [8].

## 6.3. Derleme ve Paketleme Akışı

- Kernel kaynakları GCC/NASM ile derlenir ve kernel.elf üretilir.
- HELLO kullanıcı uygulaması ayrı bağlama betiğiyle HELLO.ELF olarak oluşturulur.
- FAT32 hacmi hazırlanır ve örnek dosyalar bu hacme kopyalanır.
- Limine önyükleme dosyaları ve kernel.elf ile ISO imajı paketlenir.
- QEMU altında raw disk veya ISO üzerinden çalıştırma yapılır [5], [8].

## 6.4. Kritik Kod Parçaları ve Açıklamaları

### a) Kullanıcı sayfası eşleme mantığı (paging.c)

```
void paging_map(uint32_t virt, uint32_t phys, uint32_t flags) {
    uint32_t pd_i = (virt >> 22) & 0x3FFu;
    uint32_t pt_i = (virt >> 12) & 0x3FFu;
    uint32_t* pd = pd_ptr();

    if (!(pd[pd_i] & PAGE_PRESENT)) {
        uint32_t pt_phys = pmm_alloc_frame();
        memzero((uint8_t*)(uintptr_t)pt_phys, PAGE_SIZE);
        pd[pd_i] = (pt_phys & 0xFFFFF000u) | PAGE_PRESENT
                | PAGE_RW | (flags & PAGE_USER);
    } else if (flags & PAGE_USER) {
        pd[pd_i] |= PAGE_USER;
    }

    uint32_t* pt = (uint32_t*)(uintptr_t)(pd[pd_i] & 0xFFFFF000u);
    pt[pt_i] = (phys & 0xFFFFF000u)
            | (flags & (PAGE_USER | PAGE_RW)) | PAGE_PRESENT;
    invlpg(virt);
```

```
}
```

- İlk bölüm sanal adresi PDE/PTE indislerine ayırır ve gerekli sayfa tablosunu oluşturur.
- Kodun en kritik noktası, kullanıcı sayfası zaten var olan bir PDE içine düşerse PDE.USER bitinin ayrıca açılmasıdır.
- Son kısım PTE'yi kurar ve invlpg() ile TLB girişini geçersiz kılar; böylece yeni eşleme anında görünür olur [3], [8].

### b) Bağlam değiştirme kararı (task.c)

```
uint32_t task_maybe_switch(uint32_t cur_esp) {
    regs_t* r = (regs_t*)(uintptr_t)cur_esp;

    if (r->int_no == 32 && g_tasks[g_current].esp == 0) {
        g_tasks[g_current].esp = cur_esp;
        return cur_esp;
    }
    if (g_tasks[g_current].state == TASK_DEAD) {
        int next = pick_next_ready();
        return (next == g_current) ? cur_esp : do_switch(cur_esp, next);
    }
    if (g_yield_requested) {
        g_yield_requested = 0;
        int next = pick_next_ready();
        return (next == g_current) ? cur_esp : do_switch(cur_esp, next);
    }
    if (r->int_no != 32) return cur_esp;
    g_quantum++;
    if (g_quantum < QUANTUM_TICKS) return cur_esp;
    g_quantum = 0;
    int next = pick_next_ready();
    return (next == g_current) ? cur_esp : do_switch(cur_esp, next);
}
```

- İlk timer kesmesinde shell görevine ait ESP yakalanır ve çalışan bağlam resmileştirilir.
- Görev ölümü ve gönüllü yield istekleri, timer beklenmeden anında yeni görev seçimine götürür.
- Quantum sadece int 32 yani timer IRQ sırasında artar; bu sayede diğer kesmeler zamanlayıcı davranışını bozmaz [2], [8].

### c) Sistem çağrısı dağıtımı (syscall.c)

```
void syscall_dispatch(struct regs* rr) {
    regs_t* r = (regs_t*)rr;
    switch (r->eax) {
        case SYS_WRITE:
            r->eax = sys_write_impl((const char*)(uintptr_t)r->ebx, r->ecx);
            return;
        case SYS_EXIT:
            task_exit_current();
            r->eax = 0;
            return;
        case SYS_YIELD:
            task_request_yield();
            r->eax = 0;
            return;
        default:
            r->eax = (uint32_t)-1;
            return;
    }
}
```

- eax, hizmet numarasını; ebx/ecx ise parametreleri taşımaktadır.
- SYS_WRITE kullanıcı tamponunu terminale yazar; SYS_EXIT görevi sonlandırır; SYS_YIELD zamanlayıcıya gönüllü bırakma talebi yollar.
- Bu yapı, HELLO.ELF örneğinde görülen int 0x80 çağrılarının çekirdekte merkezi bir geçiş noktasına bağlandığını gösterir [3], [8].

# 7. Test ve Analiz

Bu bölüm, hem kaynak kod ve build artefaktlarıyla doğrudan doğrulanabilen unsurları hem de kullanıcı tarafından sağlanan QEMU ekran görüntülerinden gözlenen çalışma davranışlarını birlikte değerlendirmektedir. Böylece test bölümü yalnızca "çalışıyor" iddiasına değil; hangi işlevin hangi kanıtla desteklendiğine dayanan bir yapıya kavuşmaktadır [8].

## 7.1. Test Senaryoları

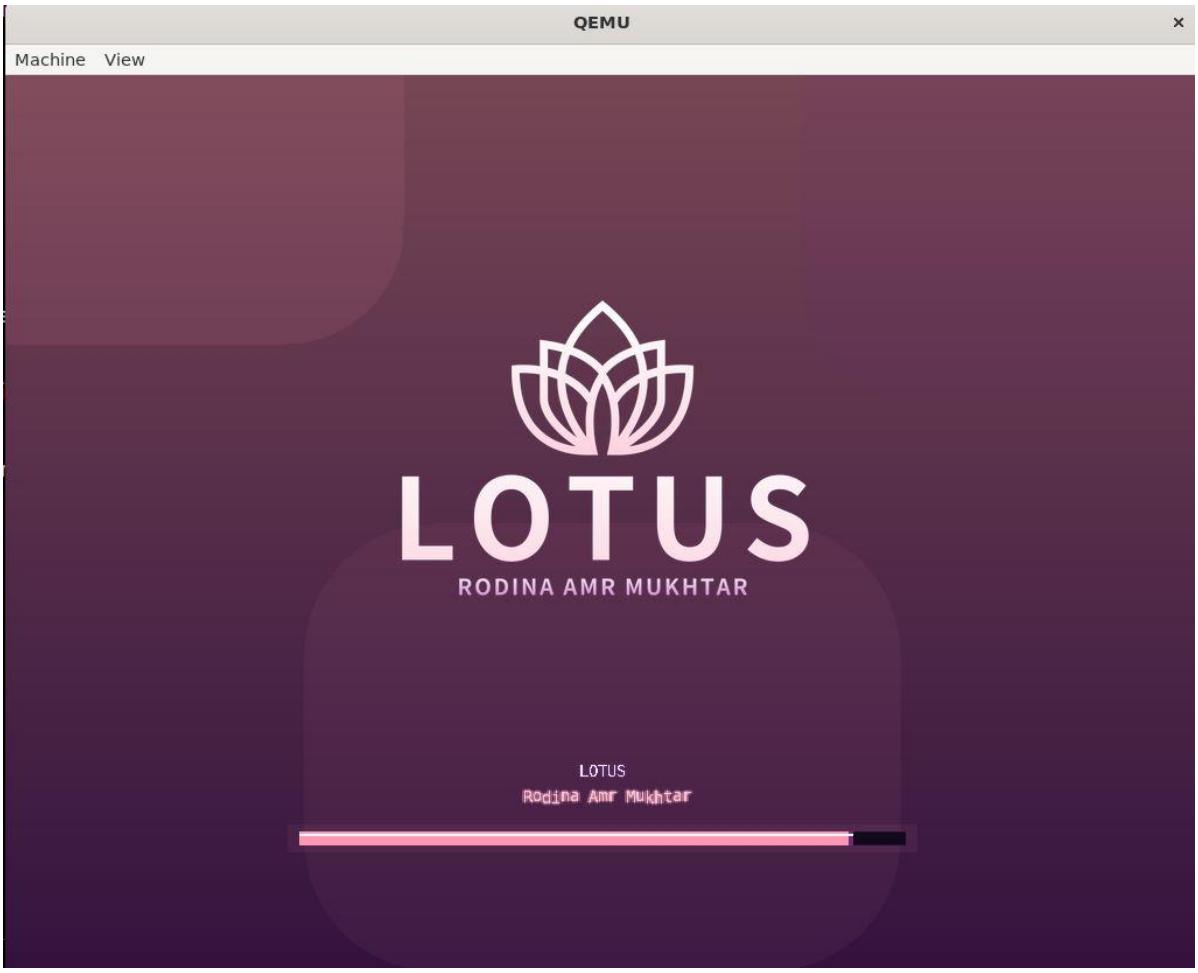
| Test senaryosu | Beklenen davranış | Kanıt / değerlendirme |
|---|---|---|
| Kernel derleme | kernel.elf üretilir | Arşivde derlenmiş artefakt mevcut [8] |
| ISO üretimi | Limine ile önyüklenebilir ISO oluşur | build/myos.iso mevcut [8] |
| Veri diski hazırlama | FAT32 hacmi ve örnek dosyalar oluşur | files/ ve build artefaktları mevcut; Explorer ekranı destekliyor [8] |
| Kabuk komutları | help, mem, ls, cat, exec vb. komutlar çalışır | Terminal ekran görüntüsü + shell.c komut tablosu [8] |
| Kullanıcı modu uygulaması | HELLO.ELF Ring3'te başlatılabilir | ELF dosyası imajda mevcut; exec komutu ve görev zinciri kodla doğrulanıyor [8] |
| GUI masaüstü | Pencereler, ikonlar, dock ve uygulama açılışı görünür | Sağlanan QEMU ekran görüntüleri ile doğrudan doğrulandı |

## 7.2. Gözlemsel Performans Verileri

| Ölçüt | Gözlenen değer | Yorum |
|---|---|---|
| GUI ilk görünür masaüstü süresi | 00:00:41 | Sistem önyükleme sonrası masaüstüne ulaşmış durumda (Şekil 6) |
| Etkileşimli terminal oturumu | 00:01:39 | help ve mem komutları aynı oturumda yürütülmüş (Şekil 7) |
| Uzun süreli GUI kararlılığı | 00:06:42 | Notepad + My Computer eşzamanlı açık; arayüz kararlı (Şekil 9) |
| Toplam kullanılabilir fiziksel RAM | 261631 KB | Terminal mem çıktısı; QEMU ortamına uygun büyüklük (Şekil 7) |
| Serbest fiziksel RAM | 226356 KB (%86.5) | Erken çalışma anında önemli rezerv kalmıştır (Şekil 7) |
| Kernel heap kullanımı | 4112 KB / 32768 KB (%12.55) | 32 MB heap içinde düşük erken kullanım (Şekil 7) |
| Scheduler quantum | yaklaşık 8.33 ms | 5 tick / 600 Hz formülü, koddan türetildi [8] |
| Paketlenmiş kullanıcı dosyaları | HELLO.ELF 4712 B; HELLO.TXT 52 B; README.TXT 33 B | My Computer görünümü FAT32 içeriğini doğruluyor (Şekil 8) |

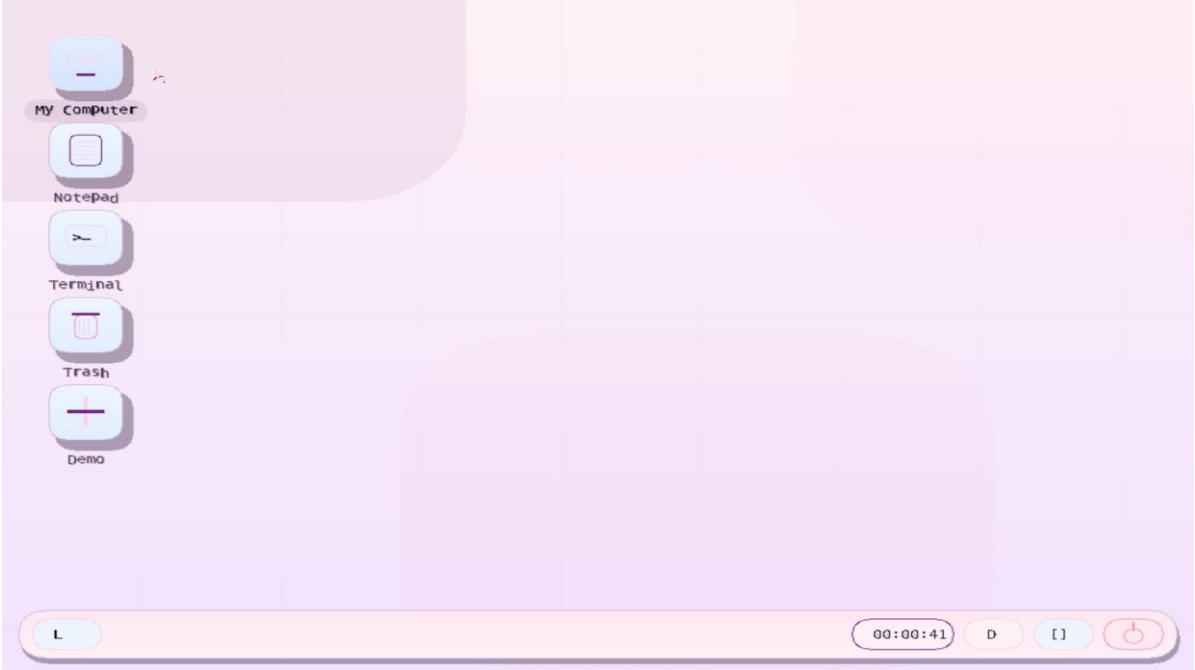
Bu veriler sentetik benchmark değil, gerçek QEMU oturumlarından gözlenen operasyonel ölçütlerdir. Yine de proje raporu açısından önemlidir; çünkü sistemin masaüstüne kadar ulaştığını, uzun süre kararlı kaldığını ve bellek tarafında erken aşamada baskı yaşamadığını göstermektedir.

## 7.3. Ekran Görüntüleri ve Açıklamalar



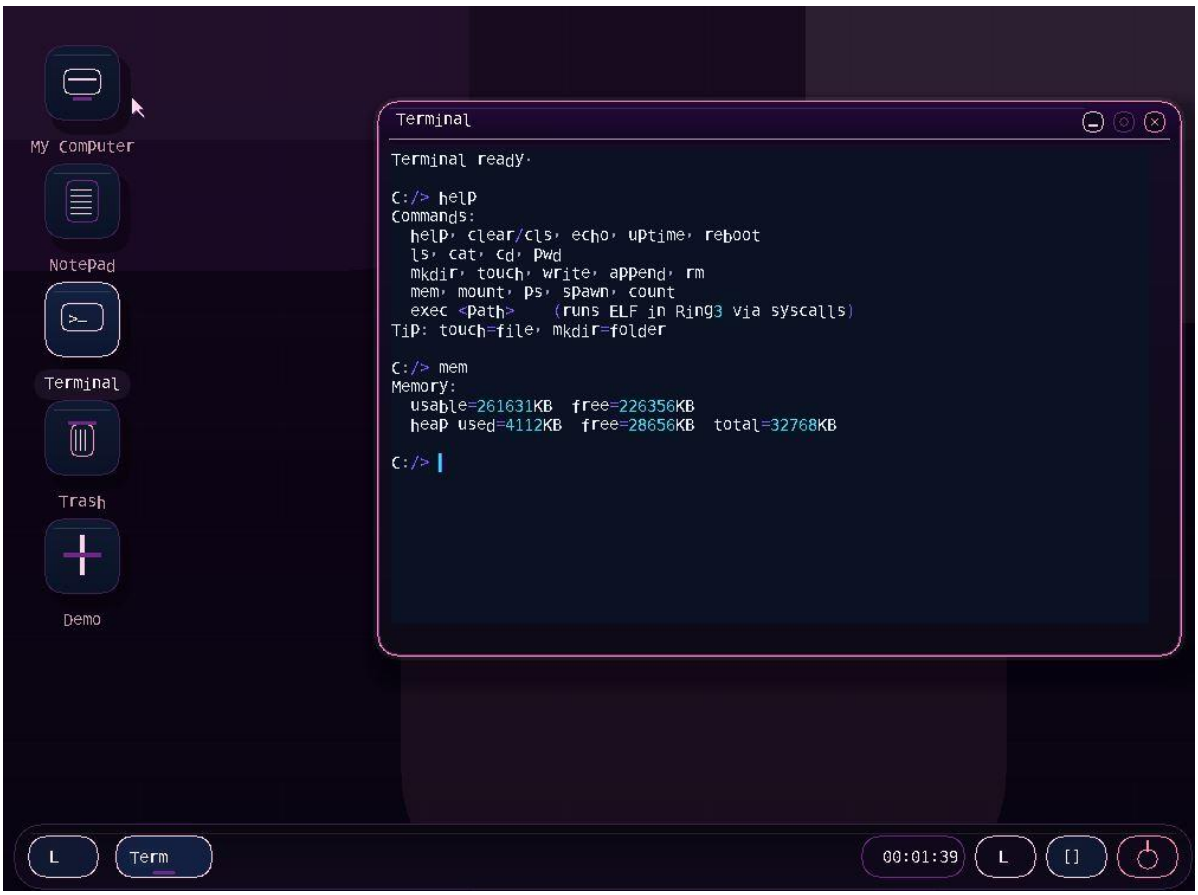

*Şekil 5. LotusOS açılış/splash ekranı*

Bu ekran, önyükleme tamamlandıktan sonra grafik katmanın devreye girdiğini ve markalı bir splash akışının gösterildiğini kanıtlamaktadır. Proje raporu açısından yalnızca işlevsel değil, kullanıcı deneyimini dikkate alan bir tasarım yaklaşımını da göstermektedir.



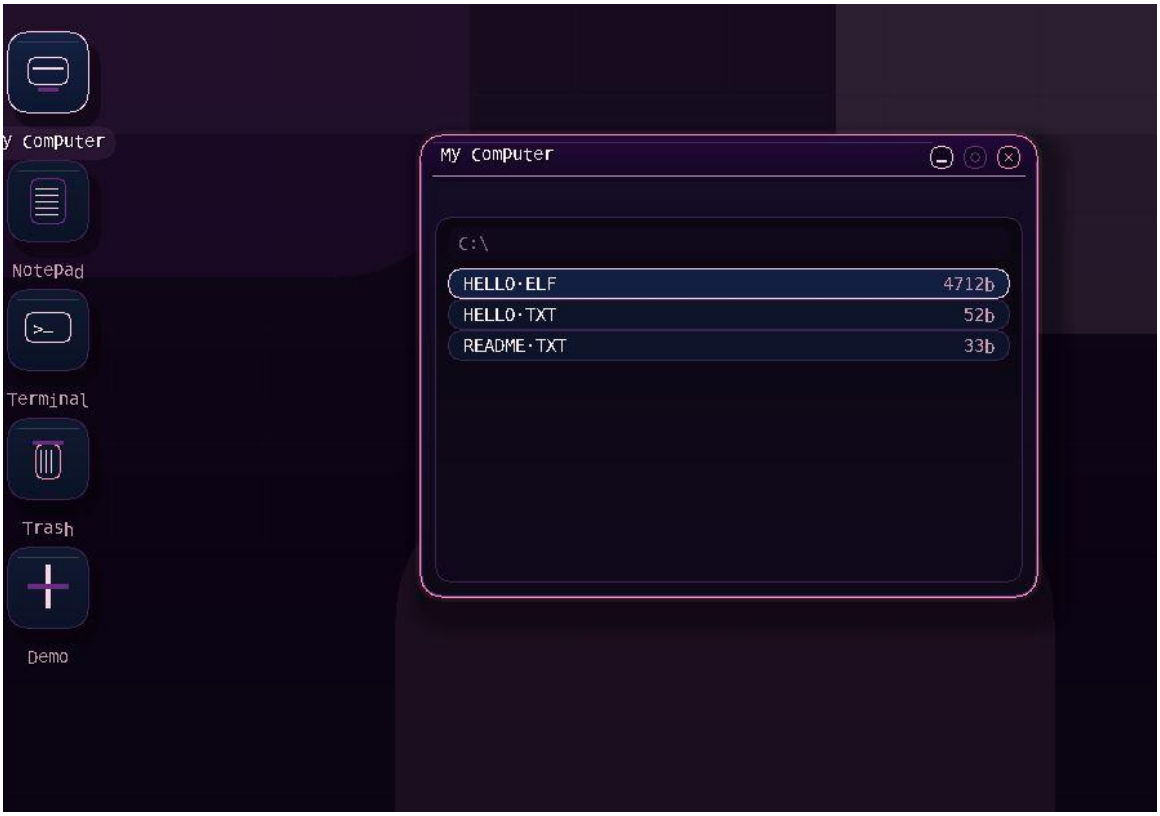

*Şekil 6. 00:00:41 anında masaüstü görünümü*

İkonlar, alt dock ve zaman göstergesiyle birlikte LotusOS masaüstü ortamı görünmektedir. Bu görüntü, çekirdek servislerinin kullanıcıya görsel bir kabuk üzerinden sunulduğunu ve GUI başlatma zincirinin başarıyla tamamlandığını göstermektedir.




*Şekil 7. Terminal penceresinde help ve mem komutları*

Terminal ekranında komut listesi, mem çıktısı ve anlık prompt birlikte görülmektedir. Bu tek görüntü hem kabuk işlevselliğini hem de PMM/kheap istatistiklerinin kullanıcıya sunulabildiğini doğrulamaktadır.




*Şekil 8. My Computer penceresinde FAT32 içeriği*

C: sürücüsünde HELLO.ELF, HELLO.TXT ve README.TXT dosyalarının listelenmesi, FAT32 bağlama zincirinin ve dosya gezgini entegrasyonunun çalıştığını göstermektedir. Özellikle HELLO.ELF dosyasının görünürlüğü, kullanıcı modu yürütüm senaryosunun paketleme yönünü desteklemektedir.

*Şekil 9. 00:06:42 anında çoklu pencere ve Notepad kullanımı*

Bu görüntüde Notepad ve dosya yöneticisinin aynı oturum içinde açık kaldığı görülmektedir. Yaklaşık yedi dakikaya yaklaşan çalışma süresi, projenin yalnızca anlık önyükleme gösterimi olmadığını; etkileşimli kullanım altında da kararlı kaldığını göstermektedir.

# 8. Karşılaşılan Zorluklar ve Çözümler

| Zorluk | Sorunun etkisi | Çözüm / mühendislik kararı |
|---|---|---|
| Framebuffer bölgesinin ayrılmaması | PMM yanlışlıkla görüntü belleğini çerçeve olarak tahsis edebilirdi | pmm_init() içinde fb_addr..fb_end aralığı reserve_range() ile ayrıldı [8] |
| PDE.USER bitinin eksik kalması | Ring3 tarafında koruma ihlali ve page fault riski oluşturdu | paging_map() içinde mevcut PDE yeniden kullanılsa bile PAGE_USER yayılımı eklendi [3], [8] |
| FAT32 bölüm konumunun değişken olması | Disk düzenine göre bağlama başarısız olabiliyordu | Önce MBR, sonra LBA 2048, ardından LBA 0 denenerek toleranslı mount zinciri kuruldu [6], [8] |
| Mouse olay taşması / senkron kaybı | İmleç sıçraması ve gereksiz yeniden çizim doğurabiliyordu | events.c tarafında mouse/tick birleştirme ve batch işleme mantığı kullanıldı [8] |
| Aşırı yeniden çizim | GUI tarafında flicker ve gereksiz maliyet oluşturabilirdi | Backbuffer, kısmi repaint ve cursor-only güncelleme yaklaşımı benimsendi [8] |
| Ring3 -> Ring0 stack sürekliliği | Sistem çağrısı ve kesmelerde yanlış kernel stack kullanımı riski | Her bağlam değişiminde TSS.esp0 güncellenerek güvenilir geçiş sağlandı [3], [8] |

Bu zorluklar, projenin yüzeyde yalnızca çalışan bir demo olmadığını; gerçek işletim sistemi geliştirme süreçlerinde karşılaşılan bellek güvenliği, kip geçişi, olay yoğunluğu ve depolama tutarlılığı gibi meselelerle uğraşıldığını göstermektedir. Akademik değerlendirme açısından bu bölüm, “ne yapıldı?” kadar “hangi hata ile neden karşılaşıldı ve nasıl çözüldü?” sorularına da yanıt vermektedir [8].

# 9. Sonuç ve Kişisel Değerlendirme

Bu projeyi incelediğimde, LotusOS'un ders ölçeğinde hazırlanmış sıradan bir çekirdek denemesinden daha ileri bir noktada olduğunu gördüm. Önyükleme, bellek yönetimi, kesmeler, görev sistemi, dosya sistemi ve grafik masaüstü gibi birbirinden çok farklı alt sistemlerin aynı proje içinde tutarlı biçimde bir araya getirilebilmesi, işletim sistemi tasarımının parçalı değil bütüncül düşünülmesi gerektiğini açık biçimde gösterdi [1], [2], [8].

Kişisel olarak bu çalışmadan en güçlü şekilde öğrendiğim nokta, işletim sistemi geliştirmede küçük görünen ayrıntıların aslında kritik öneme sahip olduğudur. Örneğin PDE.USER bitinin eksik kalması, framebuffer belleğinin ayrılmaması veya TSS.esp0'ın güncellenmemesi gibi ayrıntılar teoride küçük görünse de pratikte sistemin çökmesine ya da güvenilirliğini kaybetmesine neden olabilmektedir. Bu nedenle çekirdek geliştirme, yalnızca büyük mimari kararlar değil; bit seviyesindeki doğruluk disiplini gerektiren bir mühendislik alanıdır [3], [8].

Eğer projeyi bir sonraki aşamaya taşıma fırsatım olsaydı, süreç başına ayrı adres uzayı, demand paging, daha güçlü kullanıcı/çekirdek izolasyonu, daha gelişmiş dosya sistemi özellikleri, pencere yöneticisi için daha zengin widget seti ve gerçek benchmark ölçümleri eklemek isterdim. Buna rağmen mevcut hâliyle LotusOS; hem teknik derinlik hem de görsel sunum gücü açısından sınıf içinde güçlü bir proje raporu ve uygulama örneği oluşturmaktadır.

# 10. Kaynakça

[1] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Harlow: Pearson, 2015.

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[3] Intel Corporation, Intel 64 and IA-32 Architectures Software Developer's Manual, vols. 1-3.

[4] Free Software Foundation, The Multiboot2 Specification, Version 2.0.

[5] Limine Bootloader Project, Limine and Limine Boot Protocol Documentation.

[6] Microsoft, Extensible Firmware Initiative FAT32 File System Specification, Version 1.03.

[7] OSDev Wiki, Paging, Interrupt Descriptor Table, Task State Segment, FAT and related kernel development articles.

[8] LotusOS kaynak kod arşivi, kullanıcı tarafından sağlanan proje dosyaları, build artefaktları ve QEMU ekran görüntüleri, Mart 2026.